

REPORT ASSIGNMENT 4

PRE TRAIN THE MODEL

For training I used hyperparameters like embedding dim = 300, hidden dim = 300, batch size = 32.

```
● (elmo) sriharshitha.mareddy@gnode087:~/4$ python3 elmos.py
cuda
preprocessing done
dataset created
data loader done
Epoch: 1, Step: 1, Loss: 20.480337142944336
Epoch: 1, Step: 1001, Loss: 8.065778732299805
Epoch: 1, Step: 2001, Loss: 8.212934494018555
Epoch: 1, Step: 3001, Loss: 7.401327133178711
(2m 49s) Epoch: 1/10, Loss: 8.116306466420491
Epoch: 2, Step: 1, Loss: 7.032418251037598
Epoch: 2, Step: 1001, Loss: 7.219363212585449
Epoch: 2, Step: 2001, Loss: 6.532749652862549
Epoch: 2, Step: 3001, Loss: 6.38873291015625
(5m 40s) Epoch: 2/10, Loss: 6.692266316350301
Epoch: 3, Step: 1, Loss: 6.246681213378906
Epoch: 3, Step: 1001, Loss: 6.323758125305176
Epoch: 3, Step: 2001, Loss: 5.955044746398926
Epoch: 3, Step: 3001, Loss: 6.538154125213623
(8m 30s) Epoch: 3/10, Loss: 6.141497625478109
Epoch: 4, Step: 1, Loss: 5.454413414001465
Epoch: 4, Step: 1001, Loss: 5.868736267089844
Epoch: 4, Step: 2001, Loss: 6.184530258178711
Epoch: 4, Step: 3001, Loss: 5.686520099639893
(11m 20s) Epoch: 4/10, Loss: 5.780197749710083
Epoch: 5, Step: 1, Loss: 4.860079765319824
Epoch: 5, Step: 1001, Loss: 5.037498950958252
Epoch: 5, Step: 2001, Loss: 5.209720611572266
Epoch: 5, Step: 3001, Loss: 5.614052772521973
(14m 11s) Epoch: 5/10, Loss: 5.508107110850016
Epoch: 6, Step: 1, Loss: 5.412374496459961
Epoch: 6, Step: 1001, Loss: 5.763971328735352
Epoch: 6, Step: 2001, Loss: 5.56964111328125
Epoch: 6, Step: 3001, Loss: 5.155344009399414
(17m 1s) Epoch: 6/10, Loss: 5.288708777109782
Epoch: 7, Step: 1, Loss: 4.793088912963867
Epoch: 7, Step: 1001, Loss: 5.378584384918213
Epoch: 7, Step: 2001, Loss: 5.138951301574707
Epoch: 7, Step: 3001, Loss: 5.302000999450684
(19m 54s) Epoch: 7/10, Loss: 5.1054773499806725
Epoch: 8, Step: 1, Loss: 4.916105270385742
Epoch: 8, Step: 1001, Loss: 4.830286026000977
Epoch: 8, Step: 2001, Loss: 4.878652095794678
Epoch: 8, Step: 3001, Loss: 5.05523157119751
(22m 47s) Epoch: 8/10, Loss: 4.9482270511627195
Epoch: 9, Step: 1, Loss: 4.41542911529541
Epoch: 9, Step: 1001, Loss: 4.430171966552734
Epoch: 9, Step: 2001, Loss: 4.918301582336426
Epoch: 9, Step: 3001, Loss: 5.126290321350098
(25m 40s) Epoch: 9/10, Loss: 4.811933428382874
Epoch: 10, Step: 1, Loss: 4.417681694030762
Epoch: 10, Step: 1001, Loss: 4.559978485107422
Epoch: 10, Step: 2001, Loss: 4.529692649841309
Epoch: 10, Step: 3001, Loss: 4.751190185546875
(28m 34s) Epoch: 10/10, Loss: 4.691731185913086
model saved
```

DOWNSTREAM TASK

For classification the hyperparameters I used are embedding_dim = 600, hidden_dim = 300, num_layers(hidden) = 1, bidirectional = true, activation function = ReLU, learning rate = 0.001

1. Trainable Lambdas

- Lambda values

```
trained lambda: Parameter containing:
tensor([0.5267, 0.0809, 0.7102], device='cuda:0', requires_grad=True)
```

- On train, val set

```
Epoch: 0, Step: 1, Loss: 1.3900176286697388
Epoch: 0, Step: 1001, Loss: 0.20251095294952393
Epoch: 0, Step: 2001, Loss: 0.3242811858654022
0m 58.83485436439514s Epoch: 0, Train Accuracy: 0.87703125, Val Accuracy: 0.8924583333333334, Train Loss: 0.3489087009007732, Val Loss: 0.3046135526498159
Epoch: 1, Step: 1, Loss: 0.35903385281562805
Epoch: 1, Step: 1001, Loss: 0.3129047453403473
Epoch: 1, Step: 2001, Loss: 0.39185550808906555
1m 52.9271726682764s Epoch: 1, Train Accuracy: 0.8981145833333334, Val Accuracy: 0.89525, Train Loss: 0.2858910648537179, Val Loss: 0.2920138212492069
Epoch: 2, Step: 1, Loss: 0.2690290892783247
Epoch: 2, Step: 1001, Loss: 0.89355715662240982
Epoch: 2, Step: 2001, Loss: 0.08729642629623413
2m 48.54866552352905s Epoch: 2, Train Accuracy: 0.9110208333333333, Val Accuracy: 0.89425, Train Loss: 0.24848734365838268, Val Loss: 0.2863606052597364
Epoch: 3, Step: 1, Loss: 0.13988693058498753
Epoch: 3, Step: 1001, Loss: 0.23105426132670986
Epoch: 3, Step: 2001, Loss: 0.36780664324760437
3m 44.277507305145264s Epoch: 3, Train Accuracy: 0.9252291666666667, Val Accuracy: 0.9037083333333333, Train Loss: 0.2113860564691325, Val Loss: 0.2750873775755366
Epoch: 4, Step: 1, Loss: 0.3104865550994873
Epoch: 4, Step: 1001, Loss: 0.02054239623248577
Epoch: 4, Step: 2001, Loss: 0.2732474207878113
4m 39.974855184555054s Epoch: 4, Train Accuracy: 0.9420208333333333, Val Accuracy: 0.901375, Train Loss: 0.16419265736918898, Val Loss: 0.2981269437993566
Epoch: 5, Step: 1, Loss: 0.10405843704938889
Epoch: 5, Step: 1001, Loss: 0.08989553898572922
Epoch: 5, Step: 2001, Loss: 0.14053146541118622
5m 30.9097743034363s Epoch: 5, Train Accuracy: 0.9582395833333334, Val Accuracy: 0.9013333333333333, Train Loss: 0.12038447216432542, Val Loss: 0.3248205626110236
Epoch: 6, Step: 1, Loss: 0.10131683200597763
Epoch: 6, Step: 1001, Loss: 0.0668833926320076
Epoch: 6, Step: 2001, Loss: 0.06609147042036057
6m 31.25984263420105s Epoch: 6, Train Accuracy: 0.9693645833333333, Val Accuracy: 0.8932916666666667, Train Loss: 0.08783271100538938, Val Loss: 0.38570416129380464
Epoch: 7, Step: 1, Loss: 0.030753646045923233
Epoch: 7, Step: 1001, Loss: 0.08217040449380875
Epoch: 7, Step: 2001, Loss: 0.02039092779159546
7m 20.858006715774536s Epoch: 7, Train Accuracy: 0.97509375, Val Accuracy: 0.8962916666666667, Train Loss: 0.06987745267827995, Val Loss: 0.4328846698421985
Epoch: 8, Step: 1, Loss: 0.0911559009502138
Epoch: 8, Step: 1001, Loss: 0.01118257789105177
Epoch: 8, Step: 2001, Loss: 0.09199044853448868
8m 22.537731409072876s Epoch: 8, Train Accuracy: 0.9794479166666666, Val Accuracy: 0.8937083333333333, Train Loss: 0.05753864071471616, Val Loss: 0.4837229157057591
Epoch: 9, Step: 1, Loss: 0.02627677284181118
Epoch: 9, Step: 1001, Loss: 0.04885730892419815
Epoch: 9, Step: 2001, Loss: 0.534952017326355
9m 18.093294620513916s Epoch: 9, Train Accuracy: 0.98196875, Val Accuracy: 0.8919583333333333, Train Loss: 0.05180973346722506, Val Loss: 0.5115039674875637
```

```
Train Accuracy: 0.979125
Train Precision: 0.979144304537789
Train Recall: 0.979125
Train F1 Score: 0.9791274044907076
Train Confusion Matrix:
[[23324  93  211  259]
 [  76 23881  20   20]
 [ 168  27 23318  525]
 [ 181  30  394 23473]]
```

```
Val Accuracy: 0.8914166666666666
Val Precision: 0.8932418315343948
Val Recall: 0.8914166666666666
Val F1 Score: 0.8915268892849052
Val Confusion Matrix:
[[5505  106  192  310]
 [ 127 5729  53   94]
 [ 280  47 4868  767]
 [ 224  47  359 5292]]
```

- On test set

```

Test Accuracy: 0.8851315789473684
Test Precision: 0.8874616499906425
Test Recall: 0.8851315789473684
Test F1 Score: 0.8852849227690102
Test Confusion Matrix:
[[1707  32  63  98]
 [  42 1801  21  36]
 [   88  16 1526 270]
 [   73  16  118 1693]]

```

2. Frozen Lambdas

- Lambda values

```

trained lambda: Parameter containing:
tensor([0.4185, 0.1735, 0.2341], device='cuda:0')

```

- On train, val set

```

Epoch: 1, Step: 1, Loss: 1.3992339372634888
Epoch: 1, Step: 1001, Loss: 0.26793670654296875
Epoch: 1, Step: 2001, Loss: 0.29770785570144653
(0.0m 53s) Epoch: 1/10, Train Accuracy: 0.8643229166666667, Val Accuracy: 0.88925, Train Loss: 0.38294771977265674, Val Loss: 0.3175776062508424
Epoch: 2, Step: 1, Loss: 0.37388134002685547
Epoch: 2, Step: 1001, Loss: 0.4160834848880768
Epoch: 2, Step: 2001, Loss: 0.26834967732429584
(1.0m 46s) Epoch: 2/10, Train Accuracy: 0.8999479166666666, Val Accuracy: 0.8954166666666666, Train Loss: 0.2839723264736434, Val Loss: 0.2898155764242013
Epoch: 3, Step: 1, Loss: 0.25731274485588074
Epoch: 3, Step: 1001, Loss: 0.05419558286666687
Epoch: 3, Step: 2001, Loss: 0.1354835331439972
(2.0m 39s) Epoch: 3/10, Train Accuracy: 0.9194791666666666, Val Accuracy: 0.898375, Train Loss: 0.22862346475447218, Val Loss: 0.2867334555834532
Epoch: 4, Step: 1, Loss: 0.08429861813783646
Epoch: 4, Step: 1001, Loss: 0.08144573122262955
Epoch: 4, Step: 2001, Loss: 0.23149509727054865
(3.0m 33s) Epoch: 4/10, Train Accuracy: 0.9407291666666666, Val Accuracy: 0.89975, Train Loss: 0.16995831470191478, Val Loss: 0.30472525258362293
Epoch: 5, Step: 1, Loss: 0.17187979953289032
Epoch: 5, Step: 1001, Loss: 0.02219763770699501
Epoch: 5, Step: 2001, Loss: 0.20919178426265717
(4.0m 26s) Epoch: 5/10, Train Accuracy: 0.9583645833333333, Val Accuracy: 0.8956666666666667, Train Loss: 0.11906748691418519, Val Loss: 0.34545685530578096
Epoch: 6, Step: 1, Loss: 0.15865547955036163
Epoch: 6, Step: 1001, Loss: 0.03459842503070831
Epoch: 6, Step: 2001, Loss: 0.11729729175567627
(5.0m 20s) Epoch: 6/10, Train Accuracy: 0.9704583333333333, Val Accuracy: 0.8914166666666666, Train Loss: 0.08562521678799143, Val Loss: 0.3944926690769692
Epoch: 7, Step: 1, Loss: 0.043342266231775284
Epoch: 7, Step: 1001, Loss: 0.01230173371732235
Epoch: 7, Step: 2001, Loss: 0.05989988148212433
(6.0m 13s) Epoch: 7/10, Train Accuracy: 0.9760208333333333, Val Accuracy: 0.8904583333333334, Train Loss: 0.06739297842933835, Val Loss: 0.43093641333281996
Epoch: 8, Step: 1, Loss: 0.00465829152110219
Epoch: 8, Step: 1001, Loss: 0.04551565647125244
Epoch: 8, Step: 2001, Loss: 0.03220833092027933
(7.0m 6s) Epoch: 8/10, Train Accuracy: 0.9805416666666666, Val Accuracy: 0.8905833333333333, Train Loss: 0.056771326766249935, Val Loss: 0.4985119558532412
Epoch: 9, Step: 1, Loss: 0.07220125198364258
Epoch: 9, Step: 1001, Loss: 0.049929678440093994
Epoch: 9, Step: 2001, Loss: 0.020679587498307228
(7.0m 59s) Epoch: 9/10, Train Accuracy: 0.98298625, Val Accuracy: 0.8931666666666667, Train Loss: 0.047763523767581635, Val Loss: 0.5059493947649996
Epoch: 10, Step: 1, Loss: 0.004193685483187437
Epoch: 10, Step: 1001, Loss: 0.024409109726548195
Epoch: 10, Step: 2001, Loss: 0.11946053057909012
(8.0m 52s) Epoch: 10/10, Train Accuracy: 0.9852708333333333, Val Accuracy: 0.890125, Train Loss: 0.04189501269681205, Val Loss: 0.5302051922443013

```

```

Train Accuracy: 0.9852708333333333
Train Precision: 0.9852862570845037
Train Recall: 0.9852708333333333
Train F1 Score: 0.9852735191545277
Train Confusion Matrix:
[[23514  54  124  195]
 [   52 23913  18   14]
 [   123  17 23508  390]
 [   150  14  263 23651]]

```

```

Val Accuracy: 0.890125
Val Precision: 0.8937280706737801
Val Recall: 0.890125
Val F1 Score: 0.8901362853698886
Val Confusion Matrix:
[[5428  149  209  327]
 [ 105 5761  43   94]
 [  232  54 4751  925]
 [  187  47  265 5423]]

```

- On test set

```
Test Accuracy: 0.883421052631579
Test Precision: 0.8884376074171381
Test Recall: 0.883421052631579
Test F1 Score: 0.8833459683223128
Test Confusion Matrix:
[[1683  44  61 112]
 [ 27 1820  19  34]
 [ 75  23 1468 334]
 [ 57  18  82 1743]]
```

3. Learnable Function

- For learnable function I considered a neural network with a dropout layer, activation layer and then passed the concatenated embeddings to combine word representations across various layers.
- Functional layer weights

```
learnable function parameters: Parameter containing:
tensor([[ 0.0195, -0.1367, -0.2793, ...,  0.3037, -0.3800,  0.0248],
        [-0.0696,  0.1406, -0.2456, ...,  0.0255, -0.0912,  0.2048],
        [-0.0186, -0.2366, -0.1358, ...,  0.3199, -0.1906, -0.3381],
        ...,
        [-0.0187, -0.1197, -0.5112, ...,  0.3551,  0.2076,  0.1106],
        [-0.0527,  0.0608,  0.1091, ...,  0.2719,  0.0129, -0.1576],
        [ 0.0366,  0.1922,  0.0596, ..., -0.5230,  0.3162,  0.1907]],
        device='cuda:0', requires_grad=True)
```

- On train, val set

```

Epoch: 1, Step: 1, Loss: 1.418237328529358
Epoch: 1, Step: 1001, Loss: 0.6331721544265747
Epoch: 1, Step: 2001, Loss: 0.3301786184310913
(0.0m 55s) Epoch: 1/10, Train Accuracy: 0.8707083333333333, Val Accuracy: 0.8909583333333333, Train Loss: 0.3633566964281102, Val Loss: 0.3126062159041564
Epoch: 2, Step: 1, Loss: 0.17588219046592712
Epoch: 2, Step: 1001, Loss: 0.17809328436851501
Epoch: 2, Step: 2001, Loss: 0.07802043855190277
(1.0m 49s) Epoch: 2/10, Train Accuracy: 0.8939166666666667, Val Accuracy: 0.8955833333333333, Train Loss: 0.30287157245973745, Val Loss: 0.29390138772626717
Epoch: 3, Step: 1, Loss: 0.40552520751953125
Epoch: 3, Step: 1001, Loss: 0.36185789108276367
Epoch: 3, Step: 2001, Loss: 0.17381694912910461
(2.0m 43s) Epoch: 3/10, Train Accuracy: 0.8968645033333333, Val Accuracy: 0.895625, Train Loss: 0.29279478842260386, Val Loss: 0.29552517290910085
Epoch: 4, Step: 1, Loss: 0.24272282421588898
Epoch: 4, Step: 1001, Loss: 0.11445275694131851
Epoch: 4, Step: 2001, Loss: 0.23569725453853607
(3.0m 38s) Epoch: 4/10, Train Accuracy: 0.9000833333333333, Val Accuracy: 0.8898333333333334, Train Loss: 0.28328220825145645, Val Loss: 0.3096362202713887
Epoch: 5, Step: 1, Loss: 0.3765326142311096
Epoch: 5, Step: 1001, Loss: 0.3166820704936981
Epoch: 5, Step: 2001, Loss: 0.36081260442733765
(4.0m 32s) Epoch: 5/10, Train Accuracy: 0.9013958333333333, Val Accuracy: 0.8940416666666666, Train Loss: 0.2798507875514527, Val Loss: 0.29917181400954723
Epoch: 6, Step: 1, Loss: 0.4663638174533844
Epoch: 6, Step: 1001, Loss: 0.349629282951355
Epoch: 6, Step: 2001, Loss: 0.2646249234676361
(5.0m 27s) Epoch: 6/10, Train Accuracy: 0.9024166666666666, Val Accuracy: 0.897125, Train Loss: 0.27444726090629895, Val Loss: 0.29332685872415704
Epoch: 7, Step: 1, Loss: 0.29691800475120544
Epoch: 7, Step: 1001, Loss: 0.2605721354484558
Epoch: 7, Step: 2001, Loss: 0.10402631759643555
(6.0m 21s) Epoch: 7/10, Train Accuracy: 0.90396875, Val Accuracy: 0.8919583333333333, Train Loss: 0.27301530065760016, Val Loss: 0.30306202013293904
Epoch: 8, Step: 1, Loss: 0.19032442569732666
Epoch: 8, Step: 1001, Loss: 0.19119928777217865
Epoch: 8, Step: 2001, Loss: 0.35657861828804016
(7.0m 15s) Epoch: 8/10, Train Accuracy: 0.90590625, Val Accuracy: 0.897, Train Loss: 0.2663041890015205, Val Loss: 0.29297124487161635
Epoch: 9, Step: 1, Loss: 0.1263811412525177
Epoch: 9, Step: 1001, Loss: 0.32870495319366455
Epoch: 9, Step: 2001, Loss: 0.4195704460144043
(8.0m 10s) Epoch: 9/10, Train Accuracy: 0.9052291666666666, Val Accuracy: 0.8980833333333333, Train Loss: 0.2662201631857703, Val Loss: 0.29531273534397284
Epoch: 10, Step: 1, Loss: 0.15008747577667236
Epoch: 10, Step: 1001, Loss: 0.2921762764453888
Epoch: 10, Step: 2001, Loss: 0.3858965039253235
(9.0m 4s) Epoch: 10/10, Train Accuracy: 0.90853125, Val Accuracy: 0.898625, Train Loss: 0.259566726182277, Val Loss: 0.29400496169924734

```

```

Train Accuracy: 0.90853125
Train Precision: 0.9085419087796647
Train Recall: 0.90853125
Train F1 Score: 0.9084302318974423
Train Confusion Matrix:
[[21634  578   883   792]
 [ 281 23420  135   161]
 [ 731  202 20750  2355]
 [ 762  181 1720 21415]]

```

```

Val Accuracy: 0.898625
Val Precision: 0.8987884933125572
Val Recall: 0.898625
Val F1 Score: 0.8986246838304864
Val Confusion Matrix:
[[5438  167   301   207]
 [ 77 5817    56    53]
 [ 187   45 5208   522]
 [ 243   39  536 5104]]

```

- On test set

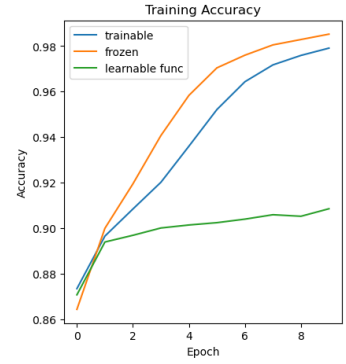
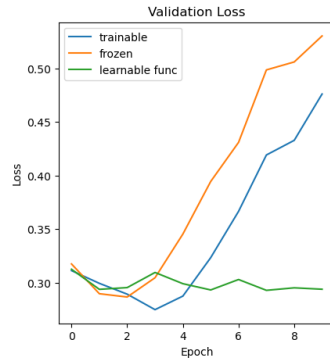
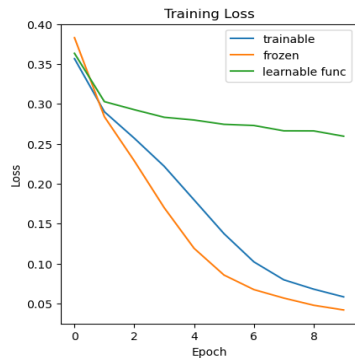
```

Test Accuracy: 0.8906578947368421
Test Precision: 0.8911083817435145
Test Recall: 0.8906578947368421
Test F1 Score: 0.8906132502961698
Test Confusion Matrix:
[[1679   61   97   63]
 [ 22 1838   32    8]
 [ 55   20 1655  170]
 [ 77   20  206 1597]]

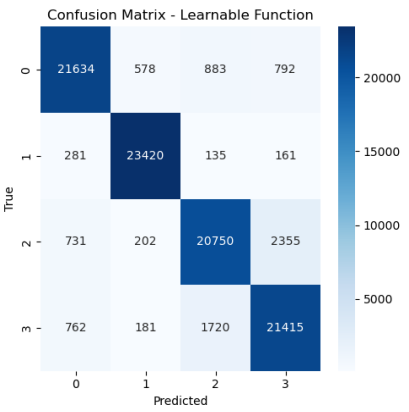
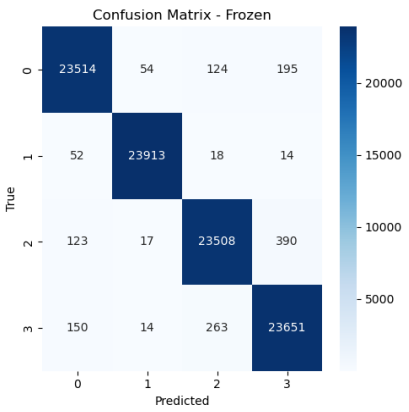
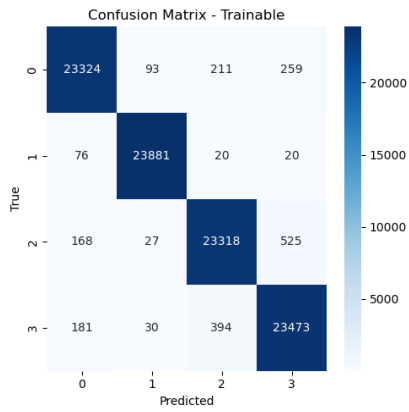
```

Comparison for different hyperparameters

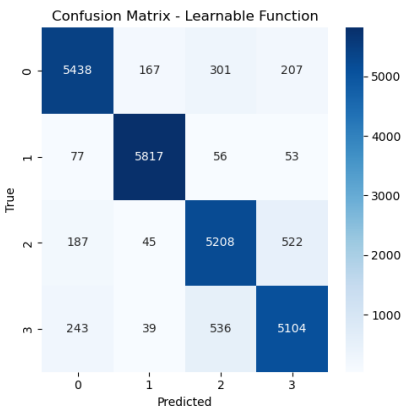
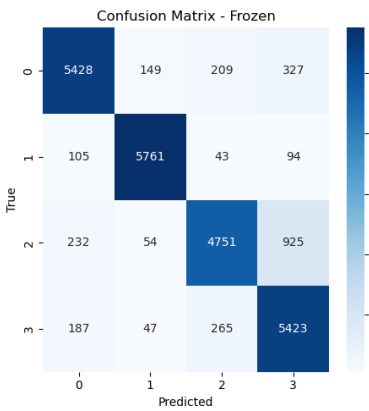
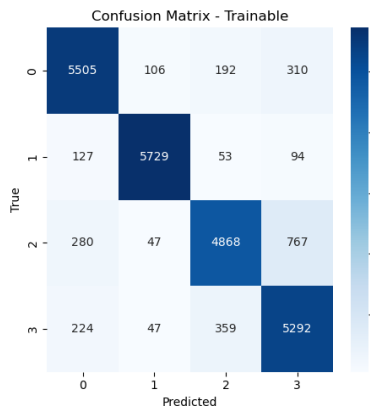
- On train, val set



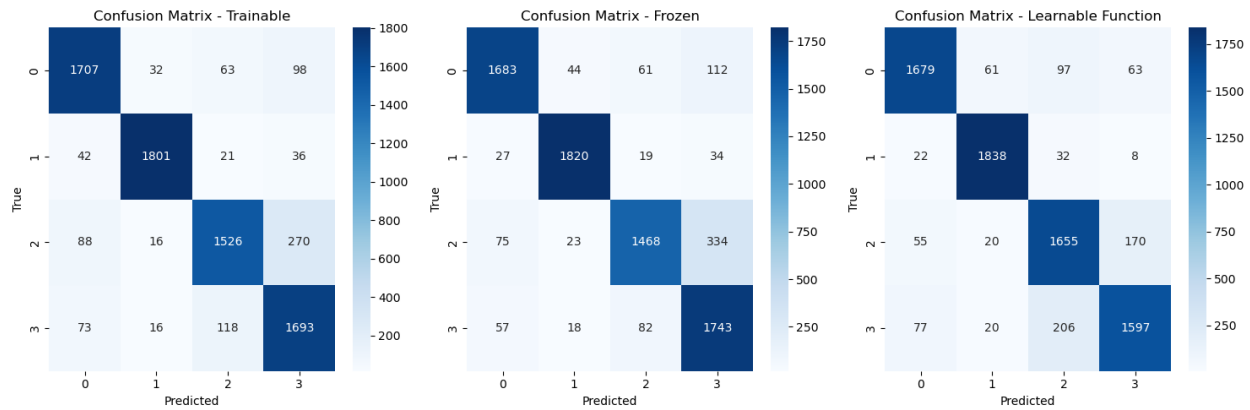
- On train



- On val



- On test



- As you can see the order of different hyperparameters for test accuracy values
 - Learnable Function > Trainable Lambdas > Frozen Lambdas

Comparison across different models

- For best model of skip-gram**

Context_window = 3

- On train,val

```
{'accuracy': 0.8740833333333333, 'f1': 0.8730298071971967, 'precision': 0.8738319111359864, 'recall': 0.8740833333333333, 'confusion_matrix': array([[21191, 1260, 924, 799],
      [ 308, 23138, 68, 190],
      [ 986, 769, 19651, 2354],
      [1376, 1205, 1849, 19932]])}
{'accuracy': 0.836375, 'f1': 0.8345848586637779, 'precision': 0.8374063512850647, 'recall': 0.836375, 'confusion_matrix': array([[4931, 364, 246, 285],
      [ 108, 6056, 45, 87],
      [ 360, 385, 4594, 901],
      [ 335, 363, 448, 4492]])}
```

- On test

```
{'accuracy': 0.8465789473684211, 'f1': 0.8451186949222905, 'precision': 0.8466953581736093, 'recall': 0.8465789473684211, 'confusion_matrix': array([[1618, 127, 75, 80],
      [ 37, 1835, 8, 20],
      [ 97, 84, 1476, 243],
      [106, 122, 167, 1505]])}
```

- For best model of svd**

Context_window = 1

- On train,val

```
{'accuracy': 0.9843125, 'f1': 0.9843082249697873, 'precision': 0.9843344044780913, 'recall': 0.9843125, 'confusion_matrix': array([[4056, 22, 22, 9],
      [ 32, 3873, 7, 5],
      [ 18, 5, 3739, 82],
      [ 6, 8, 35, 4081]])}
{'accuracy': 0.782, 'f1': 0.7831479025538817, 'precision': 0.7868997347539791, 'recall': 0.782, 'confusion_matrix': array([[912, 46, 79, 81],
      [113, 849, 34, 69],
      [ 92, 9, 593, 127],
      [ 77, 33, 112, 774]])}
```

- On test

```
{'accuracy': 0.775921052631579, 'f1': 0.7772030884998191, 'precision': 0.78276358252155, 'recall': 0.775921052631579, 'confusion_matrix': array([[1510, 78, 151, 161],  
[ 164, 1535, 69, 132],  
[ 138, 28, 1321, 413],  
[ 88, 62, 219, 1531]])}
```

We can observe that Elmo model works better than svd, skip-gram on test set.

SVD vs Skip-Gram vs ELMo:

Contextual Information: Employs deep contextualized word representations, capturing word meanings based on their context in a given sentence or document, thus offering superior contextual information compared to SVD and Skip-gram.

Nonlinear relationships: Leverages deep learning architectures, enabling it to capture intricate semantic patterns and nuances within language data, surpassing both SVD and Skip-gram in capturing non-linear relationships.

Scalability: Despite its deep architecture, ELMo can be efficiently trained on large datasets using techniques like transfer learning and pre-training, making it highly scalable.

Drawbacks:

Capturing Polysemy: Similar to Skip-gram, ELMo may struggle to effectively capture multiple meanings of polysemous words due to its single embedding per word regardless of context.

Convergence Challenges: ELMo models, like other deep learning architectures, require substantial computational resources and data for effective training, particularly for capturing nuanced contextual information.

Hyperparameter Sensitivity: Configuring hyperparameters for ELMo, such as layer depth, hidden dimension size, and training corpus selection, can significantly impact performance and may necessitate extensive experimentation.

Advantages of ELMo over SVD and Skip-gram:

Deep Contextualized Representations: ELMo generates word embeddings that are sensitive to context, allowing it to capture nuanced semantic relationships within language data more effectively than SVD and Skip-gram.

Transfer Learning: ELMo models can leverage pre-trained representations, enabling them to perform well even with limited training data, which is advantageous compared to the unsupervised nature of SVD and Skip-gram.

Flexibility: ELMo allows for the incorporation of task-specific information during fine-tuning, enhancing its adaptability and performance across various natural language processing tasks, surpassing the static embeddings produced by SVD and Skip-gram.

